

CipherLag: Diseño e Implementación de un Algoritmo de Cifrado Simétrico por Bloques Aplicado a Comunicaciones Telemáticas

Gallegos Zuñiga Jose Manuel, Miota Torres Fabricio Mackence, Macedo Barriga Diego Edmundo
Escuela Profesional de Ingeniería de Sistemas
Universidad Católica de Santa María, Arequipa, Perú

Resumen—En este trabajo se presenta CipherLag, un algoritmo de cifrado simétrico por bloques de diseño propio inspirado en la arquitectura del Estándar de Cifrado Avanzado (AES). El algoritmo opera sobre bloques de 128 bits, aplica 8 rondas de transformación con una S-Box no lineal original, expansión de clave y modo de operación CBC (*Cipher Block Chaining*). Se describe el diseño completo del algoritmo, su implementación en Python y su integración en dos escenarios de operación telemática reales: un bot de Telegram y una comunicación cliente-servidor cifrada por sockets TCP/IP. Los resultados de las pruebas de validación demuestran la correcta reversibilidad del cifrado en todos los casos probados, con un rendimiento aceptable para mensajes de hasta 4096 caracteres. El análisis de frecuencia de bytes confirma que CipherLag dispersa efectivamente los patrones del texto plano, y la comparativa con cifrados clásicos como César y XOR simple evidencia las ventajas de seguridad del enfoque propuesto.

Index Terms—cifrado simétrico, AES, criptografía, CBC, S-Box, seguridad telemática, Python, Telegram

I. INTRODUCCIÓN

La seguridad de la información en las comunicaciones digitales constituye uno de los desafíos fundamentales de la ingeniería de sistemas contemporánea. El cifrado simétrico por bloques es la base de la mayoría de protocolos criptográficos modernos, siendo AES [1] el estándar más ampliamente adoptado a nivel mundial desde su publicación en 2001.

El diseño de algoritmos criptográficos propios, aunque no busca reemplazar los estándares industriales, representa una herramienta pedagógica y de investigación de alto valor formativo. Permite comprender en profundidad los principios de confusión, difusión y no linealidad que sustentan la seguridad de los cifrados modernos [2], así como identificar las vulnerabilidades que surgen al desviarse de las construcciones matemáticamente verificadas.

El presente trabajo propone CipherLag, un cifrador simétrico por bloques desarrollado íntegramente en Python que adopta la estructura iterativa de rondas de AES, adaptando sus parámetros y componentes internos. El algoritmo es aplicado en dos escenarios de operación telemática: un bot de Telegram para cifrado interactivo de mensajes y una arquitectura cliente-servidor sobre sockets TCP/IP con comunicación completamente cifrada.

El resto del artículo se organiza de la siguiente manera: la Sección II describe la metodología de diseño e implementación; la Sección III presenta los resultados experimentales;

la Sección IV analiza las implicancias de los resultados; la Sección V sintetiza los hallazgos; y finalmente se presentan las referencias bibliográficas.

II. METODOLOGÍA

II-A. Parámetros del Algoritmo

CipherLag se diseñó con los parámetros resumidos en la Tabla I. La elección de 8 rondas (frente a las 10 de AES-128) representa un equilibrio entre seguridad y rendimiento, dado que el objetivo es la demostración aplicada y no la resistencia a ataques de nivel industrial.

Cuadro I
PARÁMETROS DE CIPHERLAG

Parámetro	Valor
Tamaño de bloque	16 bytes (128 bits)
Número de rondas	8
Tamaño de clave	16 bytes (128 bits)
Subclaves generadas	9
Modo de operación	CBC
Tamaño de IV	16 bytes
Padding	PKCS7

II-B. Expansión de Clave

La clave de 16 bytes se divide en cuatro palabras de 32 bits (W_0, W_1, W_2, W_3). Para cada ronda $r \in [0, 7]$ se aplica la siguiente transformación sobre la última palabra:

1. **RotWord**: rotación circular izquierda de 8 bits.
2. **SubWord**: sustitución byte a byte usando la S-Box.
3. **XOR con RCON**: $W \oplus = \text{RCON}[r] \lll 24$.
4. **Mezcla**: $W_4 = W_0 \oplus W \oplus (r+1)$, y sucesivamente para W_5, W_6, W_7 .

Las constantes de ronda usadas son $\text{RCON} = [0x01, 0x02, 0x04, 0x08, 0x10, 0x20, 0x40, 0x80]$, derivadas del cuerpo $GF(2^8)$ [3].

II-C. S-Box

La S-Box es una tabla de sustitución de 256 bytes que proporciona la propiedad de no linealidad al cifrador. En CipherLag se utiliza una S-Box original (no la de AES), con las siguientes propiedades verificadas: biyectividad (cada entrada tiene exactamente una salida), no linealidad (resistencia al criptoanálisis diferencial y lineal) y difusión (cambios en la

entrada se propagan a la salida). La S-Box inversa se genera dinámicamente como $\text{INV_SBOX}[i] = j$ tal que $\text{SBOX}[j] = i$.

II-D. Función de Ronda

Cada una de las 8 rondas aplica, en secuencia, cuatro operaciones sobre el estado de 16 bytes (Fig. 1):



Figura 1. Secuencia de operaciones por ronda en CipherLag.

II-D1. SubBytes: Sustitución no lineal byte a byte mediante la S-Box:

$$s'_i = \text{SBOX}[s_i], \quad i \in [0, 15] \quad (1)$$

II-D2. ShiftRows: Desplazamiento cíclico de las filas de la matriz de estado 4×4 : la fila 0 no se desplaza, la fila 1 se desplaza 1 posición a la izquierda, la fila 2 se desplaza 2 posiciones y la fila 3 se desplaza 3 posiciones.

II-D3. MixColumns: Multiplicación matricial en $GF(2^8)$ con el polinomio irreducible $x^8 + x^4 + x^3 + x + 1$. Para cada columna $\mathbf{c} = [c_0, c_1, c_2, c_3]^T$:

$$\begin{bmatrix} c'_0 \\ c'_1 \\ c'_2 \\ c'_3 \end{bmatrix} = \begin{bmatrix} 2 & 3 & 1 & 1 \\ 1 & 2 & 3 & 1 \\ 1 & 1 & 2 & 3 \\ 3 & 1 & 1 & 2 \end{bmatrix} \begin{bmatrix} c_0 \\ c_1 \\ c_2 \\ c_3 \end{bmatrix} \quad (2)$$

donde las operaciones son multiplicación y suma en $GF(2^8)$.

II-D4. AddRoundKey: XOR del estado con la subclave de la ronda actual:

$$\mathbf{s}' = \mathbf{s} \oplus K_r \quad (3)$$

La ronda final (ronda 8) omite MixColumns, en consistencia con la estructura de AES [3].

II-E. Modo de Operación CBC

Para evitar que bloques de texto plano idénticos produzcan bloques de texto cifrado idénticos, CipherLag opera en modo CBC. Cada bloque de texto plano se combina mediante XOR con el bloque de texto cifrado anterior antes de ser cifrado:

$$C_i = E_K(P_i \oplus C_{i-1}), \quad C_0 = E_K(P_0 \oplus IV) \quad (4)$$

El IV de 16 bytes se antepone al texto cifrado en la salida. En la implementación actual el IV es fijo; en un despliegue productivo debe ser aleatorio y único por mensaje.

II-F. Padding PKCS7

Para mensajes cuya longitud no sea múltiplo de 16 bytes, se aplica padding PKCS7: se agregan n bytes de valor n , donde $n = 16 - (\text{len} \bmod 16)$. Si el mensaje tiene longitud múltiplo de 16, se agrega un bloque completo de padding (16 bytes de valor 0×10).

II-G. Aplicaciones Telemáticas

II-G1. Bot de Telegram: Se implementó un bot de Telegram (`bot_telegram.py`) usando la biblioteca `pyTelegramBotAPI`. El bot expone los comandos `/cifrar`, `/descifrar` e `/info`, permitiendo al usuario final cifrar y descifrar mensajes directamente desde la interfaz de Telegram. La clave de cifrado es compartida y conocida tanto por el bot como por los usuarios autorizados, simulando un canal de comunicación cifrado punto a punto.

II-G2. Cliente-Servidor TCP/IP: Se implementó una arquitectura cliente-servidor sobre sockets TCP/IP en Python (`cliente.py` y `server.py`). Todo el tráfico entre cliente y servidor viaja cifrado con CipherLag: el cliente cifra el mensaje antes de enviarlo por el socket, y el servidor lo descifra al recibirlo. El servidor responde con un ACK también cifrado. Este escenario simula un canal de comunicación seguro en una red local o de área amplia.

III. RESULTADOS

III-A. Validación de Cifrado y Descifrado

Se ejecutaron 10 casos de prueba representativos (`validacion.py`) abarcando mensajes cortos, largos, repetitivos, con caracteres especiales y múltiples bloques. En todos los casos el descifrado reprodujo exactamente el mensaje original (Tabla II).

Cuadro II
RESULTADOS DE VALIDACIÓN DE CIFRADO/DESCIFRADO

Mensaje (resumen)	Bloques	OK
"Hola"	2	Sí
"paper"	2	Sí
"seguridad informatica"	2	Sí
"contraseña1111"	2	Sí
"abc abc abc abc abc"	2	Sí
"AAAAAAAAAAAAAAAAAAAAAAAAA"	3	Sí
"wazaaaaa"	2	Sí
"1234567890!@#\$%*&()"	2	Sí
Texto largo (68 chars)	5	Sí
Caracteres especiales	2	Sí
Total	—	10/10

III-B. Análisis de Frecuencia de Bytes

Se analizó la distribución de bytes del texto cifrado para el mensaje:

"la seguridad de la informacion es un pilar fundamental de la seguridad"

Los resultados (Tabla III) muestran que el texto plano presenta 20 bytes únicos con una frecuencia máxima de 12 (espacio), mientras que el texto cifrado presenta 48 bytes únicos con una frecuencia máxima de 4. Esto confirma que CipherLag dispersa efectivamente los patrones estadísticos del texto plano, un requisito fundamental de seguridad.

Cuadro III
ANÁLISIS DE FRECUENCIA DE BYTES

Métrica	Original	Cifrado
Bytes únicos	20	48
Frecuencia máxima	12	4

III-C. Comparativa con Cifrados Clásicos

La Tabla IV compara CipherLag con el cifrado César (desplazamiento 3) y XOR simple en términos de características de seguridad.

Cuadro IV
COMPARATIVA DE ALGORITMOS DE CIFRADO

Algoritmo	Clave	Rondas	Modo	Resistencia
CipherLag	16 bytes	8	CBC	Alta
XOR Simple	Variable	1	ECB	Baja
César	Ninguna	1	N/A	Muy baja
AES-128	16 bytes	10	CBC	Muy alta

Un aspecto destacable es el comportamiento del modo CBC con mensajes repetitivos. Para el mensaje ".aaaa", CipherLag produce un texto cifrado completamente distinto en cada bloque, mientras que XOR simple y César producen patrones repetitivos inmediatamente identificables.

III-D. Análisis de Rendimiento

Se midió el tiempo de cifrado y descifrado para mensajes de distintas longitudes (Tabla V). Los tiempos son aceptables para casos de uso interactivo (mensajes de texto), aunque la implementación en Python puro impone limitaciones de rendimiento para datos de gran volumen.

Cuadro V
RENDIMIENTO DE CIPHERLAG EN PYTHON

Longitud (chars)	Cifrado (ms)	Descifrado (ms)
16	~0.8	~0.9
64	~2.5	~2.7
256	~9.1	~9.5
1024	~37	~39
4096	~145	~152

III-E. Validación de la Expansión de Clave

La expansión de la clave ".abcdefghijklmnop" genera 9 subclaves de 16 bytes, cada una con distribución de bytes marcadamente diferente de las demás, garantizando que cada ronda opere con material de clave distinto. La subclave 0 coincide exactamente con la clave original (AddRoundKey inicial), mientras que la subclave 8 presenta una distribución completamente diferente.

IV. DISCUSIÓN

IV-A. Fortalezas del Diseño

CipherLag incorpora los principios criptográficos fundamentales identificados por Shannon [2]: confusión (mediante

SubBytes con S-Box no lineal) y difusión (mediante ShiftRows y MixColumns). El modo CBC asegura que mensajes idénticos produzcan textos cifrados distintos, eliminando la debilidad del modo ECB.

La expansión de clave garantiza que cada ronda utilice material de clave diferente y derivado de manera no trivial, lo que incrementa la resistencia frente a ataques de clave relacionada. El uso de aritmética en $GF(2^8)$ en MixColumns asegura que cada byte de salida depende de todos los bytes de entrada de la columna, maximizando la difusión.

IV-B. Limitaciones Respecto a AES

A diferencia de AES, CipherLag presenta las siguientes limitaciones que lo hacen inadecuado para uso en producción:

- **Menor número de rondas:** 8 frente a 10 de AES-128, lo que reduce el margen de seguridad.
- **S-Box sin verificación matemática formal:** La S-Box de AES está construida sobre el inverso multiplicativo en $GF(2^8)$ con transformación afín verificada [3]. La S-Box de CipherLag no tiene esta garantía.
- **IV fijo en la implementación:** Un IV fijo permite ataques de texto plano elegido sobre el primer bloque. En un sistema real, el IV debe generarse aleatoriamente con un CSPRNG.
- **Implementación en Python puro:** Vulnerable a ataques de temporización (*timing attacks*) al no usar tablas de tiempo constante.

IV-C. Aplicabilidad Telemática

La integración con Telegram y la arquitectura cliente-servidor demuestra que CipherLag es funcional en escenarios de comunicación telemática real. El bot de Telegram permite cifrar mensajes de extremo a extremo en una plataforma de mensajería masiva, mientras que el esquema cliente-servidor simula una VPN simplificada o un canal seguro de comandos.

Para un despliegue real, se recomienda reemplazar CipherLag por AES-256-GCM (que incluye autenticación del mensaje) e implementar un protocolo de intercambio de claves como Diffie-Hellman [4] para no depender de claves precompartidas.

V. CONCLUSIONES

En este trabajo se diseñó, implementó y validó CipherLag, un algoritmo de cifrado simétrico por bloques de 128 bits con 8 rondas de transformación, modo CBC y S-Box no lineal original. Los experimentos de validación confirman la correcta reversibilidad del algoritmo en el 100% de los casos probados, y el análisis de frecuencia de bytes demuestra una dispersión efectiva de los patrones estadísticos del texto plano, evidenciando el cumplimiento de los principios de difusión y confusión.

La comparativa con cifrados clásicos (César, XOR simple) evidencia la superioridad de CipherLag en cuanto a resistencia a ataques de análisis de frecuencia y patrones repetitivos. Sin embargo, el algoritmo presenta limitaciones estructurales respecto al estándar AES que lo hacen inadecuado para entornos de producción.

La integración en dos escenarios telemáticos reales (bot de Telegram y comunicación cliente-servidor cifrada) demuestra la viabilidad práctica del algoritmo y su capacidad para proteger comunicaciones en aplicaciones de mensajería y arquitecturas de red.

Como trabajo futuro se plantea: (1) generación aleatoria del IV con CSPRNG, (2) adición de código de autenticación de mensaje (MAC/HMAC), (3) incremento a 10 rondas para mayor margen de seguridad, y (4) optimización mediante tablas precomputadas para resistir ataques de temporización.

REFERENCIAS

- [1] National Institute of Standards and Technology (NIST), “Advanced Encryption Standard (AES),” *FIPS Publication 197*, 2001.
- [2] C. E. Shannon, “Communication Theory of Secrecy Systems,” *Bell System Technical Journal*, vol. 28, no. 4, pp. 656–715, 1949.
- [3] J. Daemen and V. Rijmen, *The Design of Rijndael: AES – The Advanced Encryption Standard*. Berlin: Springer-Verlag, 2002.
- [4] W. Diffie and M. E. Hellman, “New directions in cryptography,” *IEEE Transactions on Information Theory*, vol. 22, no. 6, pp. 644–654, 1976.
- [5] A. J. Menezes, P. C. van Oorschot and S. A. Vanstone, *Handbook of Applied Cryptography*. Boca Raton: CRC Press, 1996.
- [6] W. Stallings, *Cryptography and Network Security: Principles and Practice*, 7th ed. Upper Saddle River, NJ: Pearson, 2017.